

anypercent website build log (tutorial)

0) Goal

Build a site inspired by makehastecorp style with:

- portfolio-first presentation
- small product sales section
- scalable modern stack

1) Stack decision

****Chosen stack:**** Next.js (App Router) + Tailwind CSS + TypeScript

Why

- Great SEO for portfolio discoverability
- Fast and modern UI performance
- Easy to grow from portfolio into product pages/checkout later
- Component-based structure for clean iteration

2) Site structure (blueprint)

Core pages (v1)

1. `/` Home (hero + featured work + products + contact)
2. `/work` Portfolio index
3. `/work/[slug]` Case study pages
4. `/shop` Product listing
5. `/about` Brand story + capabilities
6. `/contact` Contact CTA

Homepage sections

1. ****Header/Nav****: logo, work/shop/about/contact
2. ****Hero****: strong positioning line + short value proposition
3. ****Featured Work Grid****: 3 cards with case-study links
4. ****Small Products Section****: 3 digital products with price + buy CTA
5. ****Contact Footer CTA****: handoff to contact page

3) Visual style system

Direction (inspired by reference vibe)

- Dark, high-contrast minimalism
- Tight spacing rhythm
- Lowercase typography accents
- Editorial hierarchy (big hero, compact body)
- Subtle borders/glass cards instead of heavy shadows

Tokens (v1)

- `bg`: `#0a0a0a`
- `text`: `#f4f4f5`
- `muted`: zinc-300/400/500 scale
- radius: `xl/2xl`
- border alpha: `white/10` and `white/15`

Typography

- System sans stack with Inter-first tone
- Hero: 4xl to 6xl responsive
- Body: 14–16px range for calm readability

Motion

- Introduced lightweight `animate-enter` utility (fade + rise)
- Stagger helpers: `delay-1`, `delay-2`
- Includes reduced-motion fallback for accessibility

4) Implementation details

Scaffold command

```
```bash
npx create-next-app@latest anypercent --ts --tailwind --eslint --app --src-dir --import-alias "
@/*" --use-npm --yes
```
```

Shared data model

Created `src/data/site.ts` for:

- `workItems` (portfolio entries)
- `products` (digital products + checkout links)
- `navItems`

Reusable UI

Created `src/components/site-header.tsx` for consistent nav across pages.

Pages built

- `src/app/page.tsx` (home)
- `src/app/work/page.tsx` (work index)
- `src/app/work/[slug]/page.tsx` (dynamic case study)
- `src/app/shop/page.tsx` (shop)
- `src/app/about/page.tsx` (about)
- `src/app/contact/page.tsx` (contact)

App metadata

Updated `src/app/layout.tsx` title/description for brand baseline SEO.

5) Checkout wiring approach

Products are currently wired to ****placeholder Stripe Payment Links**** in `products[].href``.

When ready, replace with real links from:

- Stripe Payment Links
- Gumroad product URLs
- LemonSqueezy checkout URLs

This keeps your UI stable while payments can be swapped without refactoring the page components

.

6) Validation gates run

Commands executed:

```
```bash
```

```
npm run lint
```

```
npm run build
```

```
```
```

Result: Passed. Routes compiled and statically generated, including dynamic work slugs.

7) Next steps (phase 3)

1. Replace placeholder work text with real projects and visuals
2. Add OG image + favicon + social metadata
3. Add analytics (Plausible or GA4)
4. Deploy to Vercel
5. Connect custom domain + email inbox for leads

8) PDF tracking

This tutorial file is the source of truth. Regenerate a new PDF snapshot after each major phase

.

Current snapshot target: ``docs/TUTORIAL-v2.pdf``